

LangChain Vs LangGraph | Agentic AI using LangGraph

1. Introduction

- This video explains the difference between **LangChain** and **LangGraph**.
- The main focus is on building **automated and agentic workflows**.
- LangGraph is useful when workflows become complex and require control, state, and reliability.

2. Goal

The goal is to understand how LangGraph helps build workflows that can handle:

- Complex control flow
- State management
- Events
- Failures
- Human intervention
- Nested workflows
- Observability

3. Disclaimer

- LangChain and LangGraph are continuously evolving frameworks.
- APIs and implementation details can change between versions.
- Focus on understanding the **core concepts**, not memorizing specific code.

4. What is LangChain?

- **LangChain** is a framework for building applications powered by LLMs.
- It provides components for:
 - Connecting LLMs
 - Prompts
 - Tools
 - Retrieval
 - Agents
 - Chains
- It makes it easier to connect different AI components together.

Simple idea: LangChain helps you **build LLM applications**.

5. LangChain Use

LangChain can be used to build applications such as:

- Chatbots

- RAG systems
- AI agents
- Question-answering systems
- Tool-using applications

A typical LangChain application may follow:

Input → LLM → Tool → LLM → Output

It is particularly useful when the workflow is relatively straightforward.

6. Understanding Automated Workflow

An **automated workflow** is a sequence of steps executed automatically.

Example:

User Request → Search → Analyze → Generate → Validate → Response

The workflow determines:

- What happens first.
- What happens next.
- Which component performs each task.
- What happens when a condition changes.

For simple workflows, a fixed sequence may be enough. Complex workflows need more flexible control.

7. Control Flow Complexity

Control flow determines how execution moves between different steps.

Simple workflow:

A → B → C → D

Complex workflow may contain:

- Conditions
- Loops
- Branches
- Parallel execution
- Retries
- Different paths depending on results

Example:

A → B → Is result good?

- Yes → C
- No → B again

LangGraph is designed to handle this type of **complex, stateful control flow**.

8. Handling State

State is the information an application needs to remember during workflow execution.

For example:

```
state = {  
  user_query,  
  search_results,  
  analysis,  
  final_answer  
}
```

Each workflow step can:

- Read the current state.
- Perform an operation.
- Update the state.
- Pass the updated state to the next step.

Simple idea: State represents **what the workflow currently knows and where it currently is**.

9. Event Driven Execution

In event-driven execution, actions happen in response to **events**.

Example:

User submits request → Event occurs → Agent starts

Another example:

Tool finishes → Event occurs → Next workflow step starts

This approach allows workflows to respond dynamically instead of simply executing everything as one fixed sequence.

10. Fault Tolerance

Fault tolerance means designing the workflow to continue operating when something goes wrong.

Possible failures:

- API failure
- Network error
- Tool failure
- Invalid output
- LLM error

A fault-tolerant workflow can use:

- Retries
- Error handling
- Checkpoints
- Alternative paths
- Recovery mechanisms

Goal: A temporary failure should not necessarily cause the entire workflow to fail.

11. Human in the Loop

Human-in-the-loop (HITL) means allowing a human to participate in the AI workflow.

Example:

AI generates action → Human reviews → Approve/Reject → Workflow continues

This is useful for sensitive or important operations such as:

- Financial transactions
- Sending important emails
- Approving AI-generated content
- Making high-impact decisions

The AI can automate most of the work while the human provides **final control when needed**.

12. Nested Workflows

A **nested workflow** is a workflow used inside another workflow.

Example:

Main Workflow

→ Research Workflow
→ Analysis Workflow
→ Writing Workflow
→ Final Response

Each smaller workflow can handle a specific responsibility.

This helps make large AI systems **modular and easier to manage**.

13. Observability

Observability means being able to see and understand what is happening inside the workflow.

You may want to know:

- Which step is currently running?
- What input did it receive?
- What output did it produce?
- Which tool was called?
- Where did an error occur?
- How long did each step take?

Observability is important for **debugging, monitoring, and improving production AI systems**.

14. Conclusion / Revision

LangChain	LangGraph
Framework for LLM applications	Framework for complex, stateful workflows
Good for chains and simpler agent applications	Good for advanced agentic workflows
Focuses on connecting components	Focuses strongly on workflow control
Can be relatively linear	Supports branching, loops, and complex execution

State management is less central	State is a core concept
Useful for building AI features	Useful for orchestrating complex AI systems

Key takeaway:

LangChain helps you build LLM applications, while LangGraph helps you control complex, stateful, agentic workflows.
